

# Eficiência de agentes de programação: qualidade, custo e latência

Isaac

**Resumo**—Este artigo delimita um protocolo para desenvolver e avaliar uma abordagem de execução de agentes de programação que melhore a relação entre custo e qualidade em comparação com um modelo em seu harness padrão, sob restrição de latência. A avaliação considera tanto economia com preservação da taxa de resolução quanto reduções de custo que justifiquem perdas delimitadas nessa taxa. Evidências sobre consumo de recursos, harnesses e validade de benchmarks fundamentam comparações pareadas com agentes de modelo único, um modelo forte com técnicas de economia de tokens e uma política simples de escalonamento. Taxa de resolução, custo monetário e tempo serão medidos separadamente, incluindo execuções malsucedidas. Roteamento entre modelos, gestão de contexto, revisão e paralelização permanecem hipóteses de mecanismo a investigar. Como ainda não há resultados experimentais, o protocolo estabelece como avaliar o benefício da abordagem, sem antecipar qual mecanismo o produzirá.

**Palavras-chave**—agentes de programação, roteamento de modelos, engenharia de software, scaffolding, avaliação de benchmarks

## I. INTRODUÇÃO

Em agentes de programação, a taxa de resolução informa se uma tarefa foi concluída segundo o verificador, mas, sozinha, não informa quanto foi necessário gastar para obtê-la. Repetições, chamadas adicionais e revisão podem alterar simultaneamente a probabilidade de sucesso e o consumo de recursos. Kapoor et al. identificam a ênfase exclusiva em acurácia como uma limitação da avaliação de agentes e formulam a otimização conjunta de acurácia e custo; seus experimentos mostram que considerar custo pode mudar a interpretação de estratégias que parecem superiores quando se observa apenas a taxa de sucesso [1]. Para decisões de engenharia, portanto, a comparação precisa mostrar a qualidade alcançada junto com os recursos consumidos, em vez de tratar qualquer ganho de resolução como vantagem suficiente.

A necessidade de avaliar qualidade e recursos conjuntamente torna o harness parte da condição experimental. Em um estudo controlado com 50 tarefas do Terminal-Bench Pro e os modelos Qwen 3.6 Plus e MiniMax M2.5, as diferenças pareadas de taxa de resolução entre harnesses variaram de 0 a 8 pontos percentuais. Os intervalos de confiança incluíram zero na maioria das comparações, o que não demonstra equivalência entre os harnesses. Mantido fixo cada modelo, o OpenCode consumiu cerca de 40 vezes mais tokens por tarefa resolvida do que o Goose. Esse contraste reforça que a taxa de resolução, isoladamente, omite diferenças substanciais de consumo mesmo em um contexto experimental controlado [2]. Como a avaliação abrangeu apenas dois modelos e três harnesses, o fator de 40 vezes demonstra a possibilidade de

grande variação, mas não estima a magnitude do efeito em outros modelos, sobretudo modelos mais recentes ou mais capazes. Tokens tampouco equivalem a um custo monetário fixo, pois tarifas, cache e categorias de cobrança variam.

Além de afetar custo e desempenho, a infraestrutura pode determinar se uma alteração produzida pelo agente chega sequer a ser testada. O Claw-SWE-Bench evidencia esse problema na interface entre agente e avaliador. Em sua configuração mínima, o modelo precisava escrever um *diff* unificado na resposta final; erros em caminhos, cabeçalhos ou linhas de contexto podiam impedir a aplicação do patch antes da execução dos testes. Na configuração completa, o agente editava os arquivos do repositório, e a camada de integração calculava o *diff* em relação ao estado inicial, removia artefatos alheios à solução e produzia o patch no formato aceito pelo avaliador. Com o mesmo modelo GLM 5.1 e as mesmas 350 tarefas, a taxa de resolução passou de 19,1% para 73,4%, enquanto a proporção de patches não aplicáveis caiu de 69,1% para menos de 1,5% [3]. O contraste não atribui esse ganho apenas à extração do patch, pois a configuração completa também modificou outros elementos do adaptador. Ele demonstra, contudo, que a pontuação pode confundir a capacidade de resolver a tarefa com a capacidade de satisfazer o contrato de entrega. A unidade de comparação deve, assim, identificar modelo, harness, ferramentas, orçamento e procedimento de entrega.

O efeito de um scaffold também pode variar com o modelo e a tarefa. Em um estudo controlado com GAIA, os efeitos diferiram entre famílias de modelos e níveis de dificuldade; a expectativa de que o scaffold se tornaria sistematicamente menos relevante com modelos mais capazes não se confirmou no nível mais difícil avaliado [4]. Esse resultado justifica medir a interação entre modelo e scaffold, em vez de pressupor que a mesma organização de agentes terá valor uniforme.

Controlar o harness ainda não basta para assegurar que a pontuação represente qualidade de engenharia de software. Gorinova et al. argumentam que benchmarks de programação podem confundir o efeito do modelo com o do harness, vincular a correção à solução de referência e fornecer pouco sinal sobre a origem das falhas [5]. Por se tratar de um artigo de posição, essas críticas não quantificam a frequência dos problemas nos conjuntos considerados aqui. Elas exigem, contudo, explicitar o que o verificador mede e registrar falhas de entrega e de execução separadamente, antes de extrapolar uma taxa de resolução para qualidade do software produzido.

Mesmo uma comparação controlada pode perder validade se o benchmark favorecer o reconhecimento de tarefas já

conhecidas. Prathifkumar et al. avaliaram uma etapa anterior à correção: a identificação dos arquivos modificados na solução de cada issue. Com apenas o texto da issue, sem acesso ao código, à lista de nomes dos arquivos ou à estrutura do repositório, Claude 3.5 e Claude 3.7 identificaram todos esses arquivos em, respectivamente, 65% e 63,2% das 500 tarefas do SWE-bench Verified. Para comparação, o estudo aplicou o mesmo teste a 500 issues de cinco repositórios Python do BeetleBox, ausentes do Verified, e a dois conjuntos mais recentes do SWE-rebench, coletados em janeiro e setembro de 2025. No BeetleBox, as taxas foram 12,2% e 12%; nos dois conjuntos do SWE-rebench, variaram de 8% a 19,27%. Trata-se de localização de arquivos, não de resolução das issues. A capacidade de localizar todos os arquivos sem consultar o repositório, somada à queda acentuada nos outros conjuntos, constitui um indício de exposição prévia às tarefas do Verified [6]. Como os conjuntos também diferem em repositórios e tarefas, essa comparação não identifica o conteúdo do treinamento nem comprova memorização de patches específicos. Ela justifica registrar a procedência e a data das tarefas, separar desenvolvimento e avaliação e interpretar com cautela pontuações em benchmarks públicos antigos.

A seleção dos benchmarks deve, portanto, combinar correspondência com a tarefa-alvo, procedência das instâncias, cobertura de contextos e verificadores reproduzíveis. O SWE-bench-Live será o conjunto principal para resolução de issues em repositórios; o Multi-SWE-bench examinará a variação entre linguagens; e o Terminal-Bench fornecerá evidência complementar sobre execução em linha de comando [7]–[9]. Esses conjuntos medem tarefas diferentes e suas pontuações serão interpretadas separadamente. A maior atualidade de um conjunto reduz uma possível fonte de exposição prévia, mas não demonstra ausência de contaminação.

Após escolher os conjuntos, a escolha de um subconjunto depende do objetivo: comparar a ordenação de configurações ou estimar a taxa de resolução no benchmark completo. Ndzomga seleciona tarefas cuja taxa histórica de sucesso está entre 30% e 70%, pois tarefas resolvidas por quase todos ou por quase ninguém oferecem pouca informação para distinguir agentes. Nos cenários avaliados, esse filtro preserva bem a ordenação, medida pelas correlações de Spearman e Kendall, mesmo quando a previsão das pontuações no conjunto completo, medida por  $R^2$ , perde precisão sob mudanças de scaffold [10]. O subconjunto pode, portanto, orientar a triagem de configurações, mas sua taxa de resolução não estima diretamente a taxa no benchmark inteiro. A avaliação confirmatória das configurações selecionadas usará tarefas reservadas, não empregadas em seu ajuste ou seleção.

O objetivo desta pesquisa é desenvolver e avaliar uma abordagem que ofereça uma relação mais favorável entre custo e qualidade do que a execução de um modelo em seu harness padrão, mantendo o aumento de latência dentro de um limite aceitável. A pergunta central é se essa abordagem reduz o custo preservando a taxa de resolução ou oferecendo uma economia que justifique uma perda delimitada nessa taxa. O protocolo compara o sistema completo com agentes de modelo

único, inclusive um modelo forte com técnicas de economia de tokens, e com uma política simples de escalonamento. A execução por modelos de menor custo com ajuda seletiva de modelos mais capazes permanece uma hipótese de solução, junto a outras intervenções de gestão de contexto e coordenação. A seção seguinte estabelece as comparações, métricas e critérios de avaliação; a investigação dos mecanismos é tratada posteriormente, sem pressupor que o roteamento ou uma arquitetura multiagente sejam necessários para obter o benefício procurado.

## II. PROTOCOLO DE AVALIAÇÃO

### A. Objetivo e desenho comparativo

A avaliação responderá se a abordagem desenvolvida oferece uma relação entre custo e qualidade mais favorável do que a execução de um modelo em seu harness padrão, sob uma restrição de latência. A referência prática será uma configuração documentada do modelo forte no harness selecionado, como Claude Code ou Codex, preservando os recursos nativos de execução. Esses nomes identificam candidatos a referência; o protocolo não pressupõe equivalência entre eles nem exige usar um modelo em um produto que não o suporte. Cada conclusão será vinculada à combinação efetivamente avaliada de modelo, harness e parâmetros.

O contraste principal será entre sistemas completos. Cada condição receberá as mesmas tarefas e estados iniciais, com permissões, acesso a ferramentas externas, testes disponíveis e tetos de recursos comparáveis. A organização interna do trabalho poderá variar, pois constitui parte da intervenção. Comparações que isolem um componente manterão os demais fatores fixos; quando isso não for possível, a diferença será atribuída ao conjunto da configuração. A ordem das execuções será aleatorizada em blocos por tarefa para reduzir a associação entre condição e horário de acesso ao serviço.

### B. Condições e contrastes

Serão avaliadas cinco condições:

- 1) **Modelo forte no harness padrão:** referência principal de resolução, custo e latência, com versão e configuração de execução registradas.
- 2) **Modelo barato no harness de referência compatível:** controle para medir o benefício e a perda de resolução obtidos pela substituição do modelo. O harness será mantido quando houver suporte; diferenças necessárias de harness serão documentadas e impedirão atribuir o contraste exclusivamente ao modelo.
- 3) **Modelo forte com economia de tokens:** o mesmo modelo e harness da primeira condição, acrescidos de técnicas explicitamente selecionadas para reduzir consumo. As técnicas serão definidas na investigação dos mecanismos e congeladas antes da avaliação final. Seu efeito será medido em custo monetário, resolução e tempo; reduzir tokens, isoladamente, não caracterizará benefício.
- 4) **Modelo barato com escalonamento simples:** execução inicial pelo modelo barato e intervenção do forte

segundo uma regra fixa, baseada apenas em sinais disponíveis durante a execução. A regra, os limites e a transferência de contexto serão documentados.

- 5) **Abordagem desenvolvida:** configuração selecionada na etapa de desenvolvimento, que poderá empregar um ou mais modelos e mecanismos de economia, verificação ou coordenação.

A quinta condição será comparada à primeira para responder à pergunta principal. Os contrastes com a segunda, a terceira e a quarta avaliarão se o benefício também pode ser obtido por alternativas mais simples. Superar apenas a referência padrão não demonstrará superioridade sobre esses controles. O contraste entre a primeira e a terceira condições medirá especificamente a contribuição das técnicas de economia aplicadas ao modelo forte.

As otimizações nativas da referência serão preservadas; a terceira condição acrescentará intervenções identificáveis sobre essa base. Todas as condições terão acesso à mesma informação externa permitida, sem testes reservados ou soluções de referência. Configurações ajustáveis receberão oportunidades documentadas de ajuste nas tarefas de desenvolvimento. Abstrações corresponderão aos componentes efetivamente adotados e às alegações sobre seus efeitos. Um controle adicional de repetição simples será incluído se repetição for um mecanismo central, distinguindo o efeito da política do benefício de gastar mais inferência [1].

### C. Benchmarks

O SWE-bench consolidou a avaliação de agentes que recebem uma issue e um repositório e devem produzir um patch verificável por execução de testes [11]. O SWE-bench-Live será o conjunto principal para esse tipo de tarefa: sua coleção atualizável reúne issues mais recentes, de um número maior de repositórios, com ambientes executáveis [7]. A atualidade reduz uma fonte potencial de exposição prévia das tarefas, mas não comprova ausência de contaminação. Tampouco uma diferença de pontuação entre conjuntos demonstra, isoladamente, memorização: a dificuldade e a composição das tarefas também podem variar.

O Multi-SWE-bench complementar a avaliação por incluir 1.632 tarefas verificadas em Java, TypeScript, JavaScript, Go, Rust, C e C++ [8]. O estudo desse conjunto relata diferenças de desempenho entre linguagens e entre métodos de agente. O SWE-PolyBench também documenta desempenho desigual e distribuição não uniforme de tarefas por linguagem, reforçando a necessidade de apresentar resultados desagregados [12]. Python aparece nas comparações publicadas do Multi-SWE-bench por meio de tarefas de outro conjunto; não será contado como uma das sete linguagens que compõem suas 1.632 instâncias.

O Terminal-Bench será a terceira avaliação, voltada à execução de tarefas por agentes em ambientes de linha de comando. Sua publicação acadêmica apresenta o Terminal-Bench 2.0 e um protocolo de verificação por tarefa [9]. O projeto mantém versões posteriores; a versão 4.0 alterou tarefas, verificadores e recursos de execução [13]. A versão e o identificador do

conjunto serão fixados antes dos experimentos. Como suas tarefas abrangem também domínios além da engenharia de software, seus resultados serão interpretados como evidência complementar sobre atuação em terminal, e não como taxa de resolução de issues.

A inclusão de cada instância exigirá ambiente reproduzível, instruções suficientes e um verificador aplicável. Auditorias do SWE-bench Verified e do SWE-bench Pro encontraram, respectivamente, problemas de exposição de soluções e de adequação dos testes, e problemas de qualidade em tarefas do conjunto Pro [14], [15]. Esses achados motivam a auditoria e o registro de problemas nas instâncias usadas; não serão extrapolados automaticamente para o SWE-bench-Live, o Multi-SWE-bench ou o Terminal-Bench. Não haverá agregação das pontuações dos três benchmarks em um escore único.

### D. Execução, entrega e registro de falhas

Cada execução começará em ambiente isolado no estado especificado pela tarefa. Nos benchmarks de issues, o agente editará os arquivos do repositório; após o encerramento, um coletor comum produzirá o patch em relação ao commit inicial, removendo apenas artefatos previamente especificados. Serão preservados a cópia de trabalho final e o patch submetido. O coletor, suas regras e a versão do avaliador serão comuns às condições, evitando introduzir exigências diferentes de serialização da solução [3]. No Terminal-Bench será aplicado o contrato de avaliação da versão selecionada.

O encerramento ocorrerá por entrega, abandono ou esgotamento de um limite de tempo, gasto ou passos. Tentativas internas e intervenções auxiliares integrarão a mesma execução e consumirão seu orçamento. Repetições experimentais começarão novamente no estado inicial e serão registradas separadamente; o melhor resultado entre repetições não será usado como resultado de uma execução comum. O avaliador reservado será acionado após o encerramento e não fornecerá sinais para novas tentativas do agente.

Ausência de entrega, patch inválido, falha nos testes e esgotamento de recursos contarão como insucesso quando decorrerem da configuração avaliada. A causa e a etapa serão registradas: localização, edição, entrega, teste, verificação, escalonamento ou parada. Falhas comprovadamente externas, como indisponibilidade de serviço ou ambiente corrompido, terão tratamento separado, segundo critérios e limite de reexecução fixados no piloto e aplicados igualmente às condições. Os registros originais e os gastos dessas ocorrências serão preservados, inclusive quando não entrarem na estimativa principal. Nenhuma falha será descartada apenas por produzir resultado desfavorável.

Serão registrados versões de modelos, harnesses e adaptadores, parâmetros de geração, prompts, permissões, limites, datas, chamadas, uso de tokens, ações de ferramenta, testes internos, decisões de escalonamento e motivos de encerramento. Procedência e datas das tarefas serão preservadas. A separação entre desenvolvimento e avaliação final impedirá que os resultados reservados sejam usados para selecionar configurações.

### E. Métricas

Para um benchmark  $b$ , seja  $I_b$  o conjunto de tarefas, com tamanho  $n_b$ , e  $K$  o número comum de repetições experimentais por tarefa e condição. Em cada execução  $r$ ,  $s_{i,c,r} = 1$  indicará aprovação pelo verificador oficial e  $s_{i,c,r} = 0$  indicará insucesso. A taxa de resolução será

$$\hat{R}_{b,c} = \frac{1}{n_b K} \sum_{i \in I_b} \sum_{r=1}^K s_{i,c,r}. \quad (1)$$

Essa taxa mede correção segundo o protocolo do benchmark. Nos conjuntos de issues, serão exigidos os testes de correção e de ausência de regressão previstos pelo respectivo avaliador [11]. A taxa de aplicação de patches e a fração de testes aprovados serão diagnósticos, sem converter aprovação parcial em resolução completa [16].

O custo de API de uma execução incluirá planejamento, execução, revisão, resumos, escalonamentos e tentativas internas, inclusive chamadas malsucedidas cobradas. Para cada chamada  $j$ , sejam  $u_{j,k}$  os tokens da categoria de cobrança  $k$ ,  $p_{m_j,k}$  o preço por milhão de tokens do modelo empregado e  $f_j$  eventuais tarifas fixas:

$$C_{i,c,r}^{\text{API}} = \sum_{j \in J_{i,c,r}} \left( \sum_k \frac{u_{j,k} p_{m_j,k}}{10^6} + f_j \right). \quad (2)$$

As categorias de entrada, cache e saída serão mutuamente exclusivas. Moeda, data, fonte das tarifas e chamadas sem informação de cobrança serão registradas; dados ausentes não serão tratados como custo zero. O custo calculado será confrontado com a cobrança quando disponível. Para referências acessadas por assinatura, eventual valoração por tarifas de API será identificada como estimativa, separada do desembolso da assinatura, e só será utilizada se houver dados suficientes de uso e preços verificáveis.

A medida monetária principal será o custo médio por tarefa, incluindo falhas,  $\bar{C}_{b,c} = \sum_{i,r} C_{i,c,r}^{\text{API}} / (n_b K)$ . A medida complementar será o custo por resolução,  $\sum_{i,r} C_{i,c,r}^{\text{API}} / \sum_{i,r} s_{i,c,r}$ , indefinido quando não houver resolução. O numerador incluirá todas as execuções. Infraestrutura e ferramentas serão informadas separadamente; uma redução apenas no custo de API não será apresentada como redução comprovada do custo operacional total. Tokens, cache, número de chamadas e participação do modelo forte serão diagnósticos de consumo.

A latência será  $T_{i,c,r} = t_{i,c,r}^{\text{fim}} - t_{i,c,r}^{\text{início}}$ , do fornecimento da tarefa ao agente até a entrega ou o encerramento, incluindo esperas, ferramentas, testes internos e coordenação. O tempo de avaliação externa será registrado à parte. A medida principal será a média  $\bar{T}_{b,c}$  sobre todas as execuções, incluindo falhas e esgotamentos de tempo; também serão apresentados mediana, percentil 95, distribuição e frequência de encerramentos por limite. A duração de trabalhos paralelos será medida pelo tempo decorrido, e seu custo incluirá todas as chamadas.

A confiabilidade da conclusão interna será descrita por

$$\hat{F}_{b,c} = \frac{\sum_{i,r} d_{i,c,r} (1 - s_{i,c,r})}{\sum_{i,r} d_{i,c,r}}, \quad (3)$$

em que  $d_{i,c,r} = 1$  indica declaração de tarefa concluída. O indicador ficará indefinido na ausência dessas declarações. Abandonos e encerramentos sem solução serão apresentados separadamente.

As expressões acima descrevem as tarefas avaliadas com pesos iguais. Uma estimativa para uma população com probabilidades desiguais de inclusão empregará os pesos do plano amostral, também nas estimativas de custo e tempo. Repetições da mesma tarefa não serão tratadas como novas tarefas independentes.

### F. Critérios de benefício e aceitação

Qualidade, custo e tempo serão analisados conjuntamente, sem redução a um escore único. Para uma configuração  $c$  e a referência forte  $c_0$ , no mesmo benchmark, definem-se a diferença de resolução  $\Delta R_c = R_{b,c} - R_{b,c_0}$ , a economia relativa  $E_c = 1 - \bar{C}_{b,c} / \bar{C}_{b,c_0}$  e a razão de latência  $L_c = \bar{T}_{b,c} / \bar{T}_{b,c_0}$ , para denominadores positivos. As estimativas desses contrastes respeitarão o pareamento por tarefa.

O protocolo adota dois cenários suficientes de benefício econômico: redução de custo com taxa de resolução preservada ou superior ( $E_c > 0$  e  $\Delta R_c \geq 0$ ); e economia de pelo menos 90% com perda de até cinco pontos percentuais ( $E_c \geq 0,90$  e  $\Delta R_c \geq -0,05$ ). Em ambos, o limite operacional inicial será  $L_c \leq 2$ . O segundo cenário corresponde, por exemplo, a reduzir o custo médio de US\$ 5 para US\$ 0,50 por tarefa, admitindo uma queda de resolução de 60% para 55%. Esses valores expressam critérios de projeto, não resultados experimentais nem limites estabelecidos pela literatura.

Os cenários não impõem economia mínima de 90% a toda configuração nem autorizam perda de cinco pontos para qualquer economia. Outras combinações serão apresentadas como trocas entre dimensões, sem aceitação automática ou interpolação de uma tolerância não especificada. Configurações dominadas, isto é, sem vantagem em nenhuma das três dimensões e piores em pelo menos uma, serão identificadas; configurações não dominadas não serão, por esse fato, consideradas aceitáveis. O limite de latência se aplica à média e será acompanhado pelo percentil 95 e pelo teto individual de execução, evitando interpretar aprovação média como garantia para cada tarefa.

Os requisitos de cada cenário deverão ser satisfeitos pela mesma configuração na mesma comparação. A análise empregará intervalos de confiança com cobertura conjunta de 95% para os contrastes confirmatórios, com procedimento de construção declarado antes da avaliação final. A aceitação exigirá que os limites inferiores sustentem os requisitos de resolução e economia e que o limite superior da razão de latência não exceda dois. Uma estimativa pontual favorável será considerada inconclusiva quanto à aceitação se sua incerteza não permitir sustentar esses requisitos. A ausência de diferença significativa não demonstrará preservação de qualidade. O cenário de perda limitada permitirá analisar não inferioridade com margem de cinco pontos, mas a aceitação exigirá também satisfazer o requisito de economia correspondente e a restrição de tempo.

Os contrastes com os demais controles serão apresentados nas mesmas dimensões. Um benefício frente ao harness padrão estabelecerá vantagem apenas frente àquela referência; vantagens adicionais atribuídas à complexidade da abordagem dependerão das comparações com economia de tokens, substituição do modelo e escalonamento simples.

#### *G. Piloto, dimensionamento e análise final*

O piloto será realizado em tarefas de desenvolvimento e terá cinco entregas: estimativas de custo e duração por condição; variabilidade entre repetições e frequência de resultados discordantes entre condições na mesma tarefa; validação da entrega e dos registros; diagnóstico de falhas externas e sinais observáveis de dificuldade; e cenários de orçamento para a avaliação final. Esses dados orientarão o tamanho da amostra, o número de repetições, os limites de gasto, tempo e passos e o tratamento de falhas externas. A regra do escalonamento simples será ajustada apenas nessa etapa. Tentativas internas, repetições experimentais e passos terão limites distintos.

A triagem de variantes poderá usar tarefas selecionadas por dificuldade histórica, conforme a distinção estabelecida na introdução [10]. Seus resultados orientarão o desenvolvimento, sem serem tratados como estimativas do benchmark inteiro. A avaliação confirmatória empregará tarefas reservadas e um plano de seleção previamente registrado. Versões, recortes e alocação entre benchmarks permanecerão decisões de dimensionamento; a seleção atual dos três conjuntos será mantida até essa definição.

O dimensionamento considerará a precisão dos contrastes pareados de resolução, custo e tempo, a dependência entre tarefas de um mesmo repositório e a variabilidade entre repetições. Antes da avaliação final serão congelados as configurações, os controles, as ablações pertinentes, os identificadores das tarefas, o plano amostral, os tetos de recursos, o método de construção dos intervalos e a regra de decisão conjunta. O piloto informará a viabilidade estatística e financeira; os critérios de aceitação não serão relaxados apenas para acomodar resultados favoráveis a uma configuração. Se houver precisão insuficiente, serão reduzidas variantes exploratórias, ampliada a avaliação ou limitada a conclusão, sem declarar benefício demonstrado com base apenas na estimativa pontual.

Os resultados serão apresentados separadamente por benchmark. No Multi-SWE-bench serão também desagregados por linguagem e repositório, acompanhados do tamanho dos grupos; esses contrastes serão descritivos quando a amostra não sustentar inferência própria. Patches, logs e diagnósticos permitirão examinar onde custo, tempo e resolução se alteraram. Uma auditoria complementar de requisitos não cobertos pelos testes oficiais só sustentará alegações adicionais se seu procedimento, amostra e critérios forem definidos previamente, com revisão sem identificação da configuração. A avaliação principal permanecerá vinculada ao verificador oficial.

### III. HIPÓTESES DE MECANISMOS PARA A ABORDAGEM

A questão de projeto é quais intervenções permitem satisfazer os critérios da Seção II. O espaço de investigação inclui economia de tokens no próprio modelo forte, execução por modelos mais baratos com assistência seletiva e mecanismos de verificação e coordenação. Essas alternativas constituem hipóteses a serem fundamentadas e testadas; o protocolo não depende de uma arquitetura específica para medir seu benefício.

A hipótese inicial de distribuição de papéis combina execução predominantemente por um modelo de menor custo com planejamento, revisão ou diagnóstico por um modelo mais capaz. Uma variante empregaria um orquestrador para acompanhar dependências, resultados intermediários, orçamento e encerramento. A ajuda do modelo forte pode orientar a execução, mas também introduzir custo e falhas de transferência de contexto. Gatilhos de escalonamento, conteúdo da assistência e limites de intervenção são decisões do mecanismo a investigar.

Outra hipótese é reduzir o consumo do próprio modelo forte por intervenções no contexto e na informação trocada durante a execução. Uma redução de tokens pode perder informações necessárias, gerar novas chamadas ou alterar a qualidade da solução. O controle com economia de tokens permitirá examinar se uma intervenção desse tipo já alcança benefício comparável ao de uma organização mais complexa; nenhuma técnica específica é assumida como eficaz nesta etapa.

A paralelização permanece uma hipótese para reduzir latência em subtarefas sem dependências conflitantes. Integração, verificação e coordenação consumirão recursos e poderão anular essa vantagem. Revisores baseados em modelo também serão investigados como mecanismos internos, mantendo a avaliação externa como medida de resolução. O benefício de revisar dependerá tanto das falhas detectadas quanto do gasto adicional e de possíveis regressões.

Permanece em aberto se a contribuição relativa do scaffold diminui com o aumento da capacidade do modelo. O estudo em GAIA não encontrou redução monotônica no nível mais difícil e observou diferenças entre famílias; seu domínio não permite antecipar a direção dessa interação nas tarefas de código aqui consideradas [4]. Essa hipótese será examinada apenas na medida em que os modelos e as ablações implementadas permitirem contrastes adequados.

A seleção dos mecanismos deverá justificar cada componente por uma hipótese de efeito sobre resolução, custo ou tempo e permitir sua remoção em ablações pertinentes. A arquitetura resultante será uma intervenção submetida ao protocolo, e sua eficiência permanecerá uma hipótese até a avaliação reservada.

### IV. DISCUSSÃO E AMEAÇAS À VALIDADE

Os controles propostos reduzem algumas fontes de confusão, mas não eliminam as limitações de exposição prévia e representatividade. Registrar datas e reservar tarefas para avaliação controla o ajuste da abordagem às instâncias observadas;

não demonstra ausência dessas tarefas no treinamento dos modelos. Diferenças entre conjuntos e linguagens também podem refletir composição de repositórios, dificuldade e domínio. A interpretação permanecerá vinculada à população efetivamente representada pelo plano amostral.

Os testes oficiais medem uma forma operacional de correção. Patches aprovados podem violar requisitos não cobertos, enquanto soluções alternativas podem ser rejeitadas por testes dependentes da implementação de referência, problemas discutidos por Gorinova et al. [5]. Registros de falhas e auditorias complementares ajudam a investigar esses casos, mas não transformam a pontuação em garantia geral de segurança ou manutenibilidade. Alegações sobre essas propriedades exigirão medidas próprias.

A referência prática será o sistema completo formado por modelo e harness. Diferenças de ferramentas internas, prompts ou gestão de contexto podem integrar a intervenção e impedir a atribuição causal a um componente isolado. As ablações permitirão restringir essa atribuição quando os demais fatores puderem ser mantidos. A comparação com o modelo forte acrescido de economia de tokens é particularmente relevante: um ganho frente ao uso padrão pode já ser alcançado por uma intervenção mais simples. Diferenças residuais de suporte entre modelos e harnesses serão limitações explícitas dos contrastes.

Modelos, serviços e tarifas podem variar durante a coleta. Versões registradas e ordem de execução aleatorizada reduzem a confusão temporal, sem garantir estabilidade do serviço. Custos de API estimados por tokens não equivalem automaticamente a despesas de assinatura ou custo operacional total. A separação desses valores delimita o significado da economia observada, inclusive quando uma redução de chamadas exige mais infraestrutura local.

O estudo controlado em GAIA encontrou efeitos de scaffold dependentes de modelo e dificuldade, inclusive sem a redução monotônica esperada para modelos mais capazes no nível difícil [4]. Nesse nível, a vantagem do scaffold com múltiplos papéis apareceu com maior clareza na família Anthropic, mas não nos modelos de outros provedores examinados. Os scaffolds estruturados também fizeram menos chamadas de ferramenta, terminaram mais rapidamente em média e recuperaram-se com maior frequência de erros intermediários; isso mostra que acrescentar etapas de organização não implica necessariamente mais ações ou maior gasto. Ainda assim, sua transferência para engenharia de software é limitada pelo domínio avaliado e pelo fato de o controle ReAct usar uma superfície de ferramentas diferente da dos outros scaffolds. Já Vats e Golev observaram grande variação em tokens por tarefa resolvida entre harnesses de programação, mas a amostra de 50 tarefas e diferenças nos controles de turnos limitam a generalização do fator relatado para custo monetário ou para os modelos futuros [2]. Portanto, resultados anteriores justificam medir a interação modelo-scaffold e o consumo efetivo, sem fixar sua magnitude no presente estudo.

A seleção de variantes com dados de desenvolvimento pode produzir configurações ajustadas às tarefas observadas.

A avaliação reservada e a distinção entre triagem e confirmação limitam esse problema. Subconjuntos escolhidos por dificuldade não sustentarão automaticamente inferência para todo o benchmark; repetir execuções sobre poucas tarefas pouco substitui diversidade de tarefas e repositórios. Quando orçamento ou variabilidade impedirem precisão suficiente, a aceitação permanecerá inconclusiva.

Os limiares de economia, resolução e latência são critérios de projeto. Uma configuração aceita segundo esses critérios pode ser inadequada em um contexto que exija preservar integralmente a resolução ou imponha prazos mais estritos. O limite de tempo médio também não garante ausência de casos muito lentos. Por isso, serão apresentados os valores separados, sua incerteza e a distribuição de duração, permitindo examinar outras preferências sem redefinir retrospectivamente o sucesso do estudo.

Por fim, resultados em um benchmark, par de modelos ou harness não serão generalizados automaticamente a outros. O protocolo mede recursos consumidos pelo agente, sem estimar o esforço de supervisão e correção humana. Eventual benefício econômico segundo as métricas adotadas não demonstrará, por si só, maior produtividade de desenvolvedores ou superioridade de uma arquitetura em todos os contextos.

## V. CONCLUSÃO

As evidências examinadas sustentam que a taxa de resolução, isoladamente, não basta para comparar agentes de programação sob restrições de engenharia. O consumo de recursos, o harness e o procedimento de entrega afetam a interpretação do desempenho; exposição prévia às tarefas, verificadores incompletos e seleção de subconjuntos limitam o alcance das conclusões. Esses fatores fundamentam a avaliação conjunta de qualidade operacional, custo e latência.

O protocolo estabelece como avaliar uma abordagem frente ao modelo no harness padrão e a controles de menor custo, economia de tokens no modelo forte e escalonamento simples. O benefício será examinado tanto pela preservação da resolução com economia quanto por uma redução substancial de custo que justifique uma perda delimitada de resolução, sob restrição de tempo. Piloto e desenvolvimento antecederão o congelamento das configurações e a avaliação reservada. A investigação dos mecanismos deverá determinar como alcançar essas condições; roteamento, gestão de contexto, revisão e paralelização permanecem hipóteses, sem superioridade experimental demonstrada.

## REFERÊNCIAS

- [1] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, “AI Agents That Matter,” arXiv:2407.01502, 2024. [Online]. Available: <https://arxiv.org/abs/2407.01502>
- [2] N. Vats and O. Golev, “The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation,” arXiv:2607.22585, 2026. [Online]. Available: <https://arxiv.org/abs/2607.22585>
- [3] M. Zheng, K. Han, B. Li, H. Xu, Y. Tian, W. He, H. Zhou, J. Guo, H. Hu, L. Ma, C. Xu, G. Dai, L. Xia, Y. Wei, Y. Wang, and Y. Wang, “Claw-SWE-Bench: A benchmark for evaluating OpenClaw-style agent harnesses on coding tasks,” arXiv:2606.12344, 2026. [Online]. Available: <https://arxiv.org/abs/2606.12344>

- [4] J. Starace, “Scaffold effects on GAIA: A controlled comparison,” arXiv:2606.08529, 2026. [Online]. Available: <https://arxiv.org/abs/2606.08529>
- [5] M. I. Gorinova, M. Baker, A. Heineke, M. Shaposhnikov, R. Willoughby, and D. Knox, “Position: Coding benchmarks are misaligned with agentic software engineering,” arXiv:2606.17799, 2026. [Online]. Available: <https://arxiv.org/abs/2606.17799>
- [6] T. Prathikumar, N. S. Mathews, and M. Nagappan, “Does SWE-Bench-Verified test agent ability or model memory?” arXiv:2512.10218, 2025. [Online]. Available: <https://arxiv.org/abs/2512.10218>
- [7] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, “SWE-bench Goes Live!” arXiv:2505.23419, 2025.
- [8] ByteDance Seed, “Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving,” arXiv:2504.02605, 2025.
- [9] M. A. Merrill *et al.*, “Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces,” arXiv:2601.11868, 2026. [Online]. Available: <https://arxiv.org/abs/2601.11868>
- [10] F. Ndzomga, “Efficient benchmarking of AI agents,” arXiv:2603.23749, 2026. [Online]. Available: <https://arxiv.org/abs/2603.23749>
- [11] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub issues?” in *International Conference on Learning Representations*, 2024.
- [12] M. S. Rashid *et al.*, “SWE-PolyBench: A multi-language benchmark for repository level evaluation of coding agents,” arXiv:2504.08703, 2025.
- [13] R. Marten, “Terminal-Bench 4.0: Calibrating task resources, fixing tasks, and removing saturated tasks,” Terminal-Bench blog, Aug. 2026. [Online]. Available: <https://www.tbench.ai/news/terminal-bench-4-0>
- [14] OpenAI, “Por que o SWE-bench Verified não mede mais capacidades de codificação de ponta?” Feb. 2026. [Online]. Available: <https://openai.com/pt-BR/index/why-we-no-longer-evaluate-swe-bench-verified/>
- [15] —, “Separando sinal de ruído em avaliações de programação,” Jul. 2026. [Online]. Available: <https://openai.com/pt-BR/index/separating-signal-from-noise-coding-evaluations/>
- [16] Q. Zhou *et al.*, “FeatureBench: Benchmarking agentic coding for complex feature development,” in *International Conference on Learning Representations*, 2026.